



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
FLUMINENSE

Sistemas de Informação

Inteligência Computacional

Eduardo Rodrigues

eduardo.rodrigues@iff.edu.br

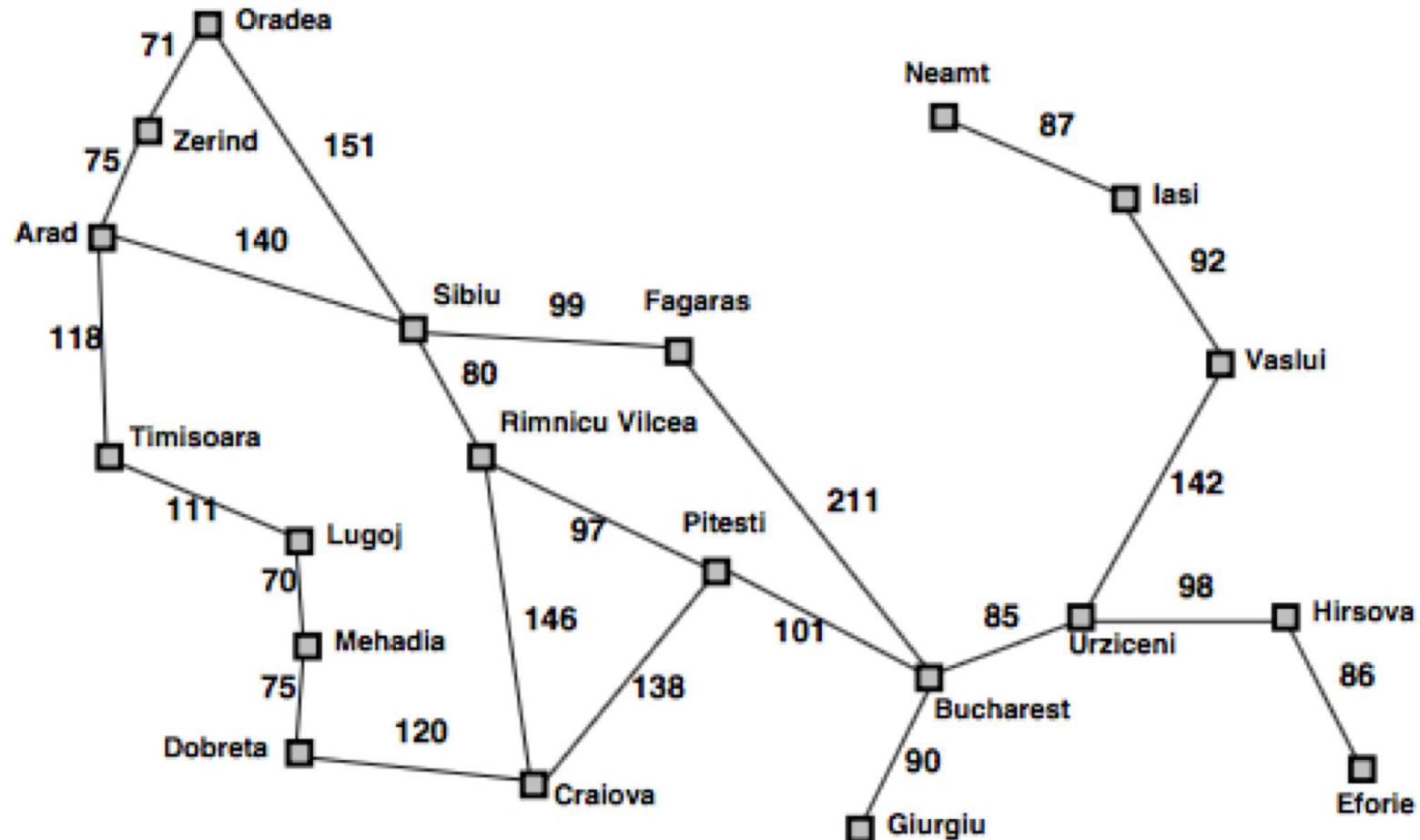
Objetivos

- ▶ Conhecer o problema de melhor rota: Arad para Bucareste;
- ▶ Entender as etapas de modelagem do problema de melhor rota entre cidades;
- ▶ Implementar o algoritmo para resolução do problema através da Busca Gulosa;

Arad para Bucureste

Arad para Bucureste

► Algoritmos de busca inteligente



Modelagem do problema

Modelagem

► Componentes de um problema

- 1. Estado inicial;
- 2. Estado final (objetivo);
- 3. Espaço de estados;
- 4. Ações para passar de um estado para outro;
- 5. Solução (caminho que leva do estado inicial ao estado final).

Modelagem

► Componentes do problema

- 1. Estado inicial = **Cidade de Arad**;
- 2. Estado final (objetivo) = **Cidade de Bucareste**;
- 3. Espaço de estados = **lista de cidades alcançáveis, a partir de uma cidade atual**;
- 4. Ações para passar de um estado para outro = **função de transição. Escolha da próxima cidade a ser visitada, baseado na heurística escolhida**;
- 5. Solução (caminho que leva do estado inicial ao estado final) = **lista de cidades visitadas, partindo de Arad e chegando a Bucareste, com o menor custo**.

Modelagem

► Heurística

- Técnica utilizada para auxiliar na resolução de problemas através de simplificações que permitem soluções aproximadas, quando a aplicação dos métodos clássicos é inviável.

► Métodos Heurísticos

► Local/Global

- Iteração de procedimentos: busca gulosa, busca A*, etc.

► Algoritmos Bioinspirados

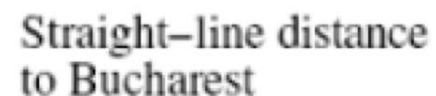
- Inspirados em fenômenos naturais: colônia de formigas, algoritmos genéticos, etc.

Modelagem

► Heurística Construtiva Gulosa

- Constrói uma solução elemento a elemento - A cada passo é adicionado um único elemento;
- O elemento escolhido é o “melhor” segundo a função objetivo/função de avaliação;
- O método termina quando todos os elementos foram inseridos na solução.

► Heurística



Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Modelagem - Implementação

► Vértice (Cidade)

- Será constituída pelos seguintes parâmetros:
 - Rótulo: nome da cidade;
 - Distância ao objetivo: distância para Bucareste a partir da cidade atual (**heurística**);
 - Visitado: status para indicar se o vértice já foi visitado;
 - Adjacentes: lista de cidades adjacentes a cidade atual.
- O construtor deverá receber apenas o rótulo e a distância como parâmetro;
- Possuía métodos para:
 - Get's e Set's;
 - Adicionar adjacente;
 - Exibir lista de vértices adjacentes.

Modelagem - Implementação

- ▶ **Adjacente (Conexão direta entre duas cidades)**
 - ▶ Será constituída pelos seguintes parâmetros:
 - ▶ Vértice: cidade;
 - ▶ Custo: distância entre as cidades conectadas diretamente.
 - ▶ Possuirá métodos para:
 - ▶ Get's e Set's.

Modelagem - Implementação

► Grafo (Mapa das cidades)

- Será constituída pelos seguintes parâmetros:
 - Vértice: todas as cidades que fazem parte do mapa;
- O construtor deve alocar memória pra todas as cidades com os seu respectivo rótulo e distância em linha reta para a cidade destino;

Modelagem - Implementação

► Grafo (Mapa das cidades)

► Exemplo de implementação

```
public class Grafo {  
  
    // Atributos são as cidades (vértices do grafo)  
    public Vertice arad, zerind, oradea, sibiu, timisoara, lugoj, mehadia,  
        dobreta, craiova, rimnicu, fagaras, pitest, bucharest, giurgiu;  
  
    /* Construtor de grafo personalizado para o problema.  
    Não há necessidade de fazer função específicas de grafos.  
    Por exemplo: inserir à esq, inserir à dir, etc.  
    */  
    public Grafo() {  
        /* Criar cidades do mapa  
        Os valores numéricos representam as heurísticas para a  
        cidade que será o destino final. Distância em linha reta.  
        */  
        arad = new Vertice( rot:"Arad", dist_obj:366);  
        zerind = new Vertice( rot:"Zerind", dist_obj:374);  
        oradea = new Vertice( rot:"Oradea", dist_obj:380);  
        sibiu = new Vertice( rot:"Sibiu", dist_obj:253);
```



```
        /* Criar ligação (arestas) entre cidades  
        Nesse caso, deve existir a conexão de ida e volta entre  
        elas. Ou seja, precisamos criar as ligações de  
        A -> B e B -> A.  
        */  
        arad.adicionaAdjacente(new Adjacente( vertice:zerind, custo:75));  
        arad.adicionaAdjacente(new Adjacente( vertice:sibiu, custo:140));  
        arad.adicionaAdjacente(new Adjacente( vertice:timisoara, custo:118));  
  
        zerind.adicionaAdjacente(new Adjacente( vertice:arad, custo:75));  
        zerind.adicionaAdjacente(new Adjacente( vertice:oradea, custo:71));  
  
        oradea.adicionaAdjacente(new Adjacente( vertice:zerind, custo:71));  
        oradea.adicionaAdjacente(new Adjacente( vertice:sibiu, custo:151));  
  
        sibiu.adicionaAdjacente(new Adjacente( vertice:oradea, custo:151));  
        sibiu.adicionaAdjacente(new Adjacente( vertice:arad, custo:140));  
        sibiu.adicionaAdjacente(new Adjacente( vertice:fagaras, custo:99));  
        sibiu.adicionaAdjacente(new Adjacente( vertice:rimnicu, custo:80));
```

Modelagem - Implementação

- ▶ **Vetor Ordenado (Lista de cidades) - USO OPCIONAL**
 - ▶ Será constituída pelos seguintes parâmetros:
 - ▶ Capacidade: número total de elementos possíveis;
 - ▶ Última posição: índice do último elemento inserido na lista;
 - ▶ Lista de vértices: lista de vértices conectadas diretamente a outro vértice.
 - ▶ O construtor deve receber a capacidade como parâmetro de entrada, inicializar o índice da última posição com -1 e alocar memória para a lista de vértices.
 - ▶ Deve conter os métodos:
 - ▶ Get's e Set's;
 - ▶ Inserção ordenada de um Vértice;
 - ▶ Imprime os vértices da lista.

Modelagem - Implementação

▶ Gulosa (Busca pelo vértice de destino)

▶ Será constituída pelos seguintes parâmetros:

- ▶ Vértice: objetivo da busca;
- ▶ Status: verifica se vértice já foi encontrado;

▶ O construtor deve receber a cidade a ser buscada. O parâmetro booleano de status deve iniciar com valor FALSE;

▶ Deverá conter método para:

- ▶ Realizar a busca de modo iterativo ou recursivo, a partir do vértice atual passado como parâmetro.

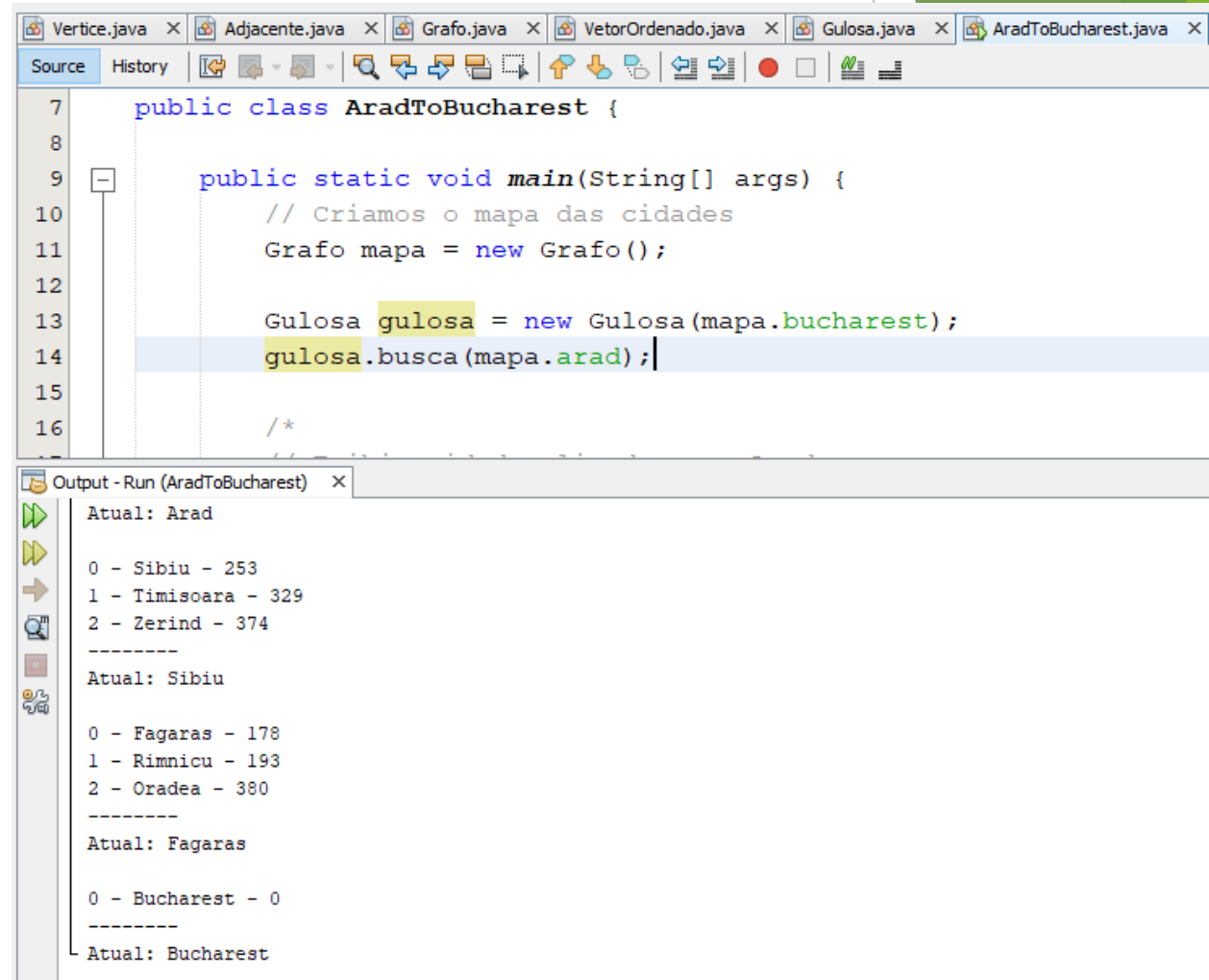
Modelagem - Implementação

► Solução para busca recursiva

- 1- Exibir o rótulo do vértice atual;
- 2- (IF) Verificar se o rótulo do vértice atual é igual ao rótulo do objetivo ou se distância ao objetivo é igual a 0;
- 3- (ELSE) Criar vetor ordenado vazio para armazenar vértices adjacentes em ordem crescente:
 - 3.1. Criamos vetor ordenado, a partir da quantidade de adjacentes do vértice atual;
 - 3.2. (LAÇO) Percorrer lista de adjacentes verificando se o vértice atual já foi visitado;
 - 3.2.1. (IF) Caso não tenha sido, *setar* como visitado e inserir no vetor ordenado.
 - 3.3. Exibir o vetor ordenado;
 - 3.4 Pegar a primeira posição do vetor ordenado, caso não seja *null*, e chamar a busca recursiva tendo esse vértice como parâmetro.

Modelagem - Implementação

- ▶ **AradToBucharest (Classe de testes)**
 - ▶ Conterá o método *main* para realização dos testes;
 - ▶ Devemos criar um objeto *Grafo* e um objeto *Gulosa*, para podermos realizar o teste de busca.



The screenshot shows a Java IDE with several open files: *Vertice.java*, *Adjacente.java*, *Grafo.java*, *VetorOrdenado.java*, *Gulosa.java*, and *AradToBucharest.java*. The *AradToBucharest.java* file is active, displaying the following code:

```
7 public class AradToBucharest {  
8  
9     public static void main(String[] args) {  
10         // Criamos o mapa das cidades  
11         Grafo mapa = new Grafo();  
12  
13         Gulosa gulosa = new Gulosa(mapa.bucharest);  
14         gulosa.busca(mapa.arad);  
15  
16         /*
```

Below the code editor, the *Output - Run (AradToBucharest)* window shows the execution results:

```
Atual: Arad  
0 - Sibiu - 253  
1 - Timisoara - 329  
2 - Zerind - 374  
-----  
Atual: Sibiu  
0 - Fagaras - 178  
1 - Rimnicu - 193  
2 - Oradea - 380  
-----  
Atual: Fagaras  
0 - Bucharest - 0  
-----  
Atual: Bucharest
```

Bibliografia Básica

- ▶ FERNANDES, Anita Maria da Rocha. **Inteligência artificial:** noções gerais. Florianópolis: Visual Books, 2008.
- ▶ RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial**, tradução da 2a edição, Elsevier, Rio de Janeiro, 2004.
- ▶ COSTA, Ernesto; SIMOES, Anabela. **Inteligência Artificial - Fundamentos e Aplicações**. Editora FCA, 2a Edição, 2008.

Bibliografia Complementar

- ▶ MONTGOMERY, Eduard. **Redes Neurais - Fundamentos e Aplicações Com Programas Em C.** Editora Ciência Moderna, 2007.
- ▶ SHAW, I. S.; SIMÕES, M. G. - **Controle e Modelagem Fuzzy**, Editora Edgard Blucher Ltda, 1ª. Edição, 2001.
- ▶ CRUZ, Leôncio T., ALENCAR, Antonio J., SCHMITZ, Eber Assis. **Assistentes Virtuais Inteligentes - Conceitos e Estratégias.** Editora Brasport, 2013.
- ▶ DE CAMPOS, Mario M.; SAITO, Kaku. **Sistemas Inteligentes Em Controle e Automação de Processos .** Editora Ciência Moderna, 2004.
- ▶ CAWSEY, Alison. **The Essence of Artificial Intelligence**, Prentice Hall, 1998.

Obrigado!

► Até o próximo encontro.